STRUKTUR OOP (1)

- Pemrograman **Terstruktur/Prosedural** hanya memiliki sebuah struktur yaitu anatomy.
- OOP memiliki **dua struktur** yaitu Taxonomy (Gen-Spec) dan Anatomy (Whole-Part).
- Project sebagai suatu solusi OOP dari suatu masalah tersusun atas struktur Taxonomy dari kumpulan class.

STRUKTUR OOP (2)

- Class-class dalam satu project membentuk satu struktur pohon/hirarki **piramida taxonomy** tersendiri yg khas untuk problem tersebut.
- Setiap class memiliki struktur **anatomy** tersendiri yang terdiri atas tiga komponen utama yaitu **attribute**, **method()**, dan **constructor()**, komponen lain yaitu objek, modifier, dan inner-class.

STRUKTUR OOP (3)

- Satu editor akan digunakan untuk ngoding sebuah class, sehingga class-class dibangun, compile dan run satu per satu (tidak sekaligus satu program/project dalam satu editor seperti pemrograman terstruktur).
- Objek-objek yang dicetak dengan class yang sama dapat memiliki ke**sama**an **ciri (attribute)** misal sebagai Manusia dan ke**sama**an **perilaku yg khas (method)** milik class Manusia pada umumnya.

Contoh Struktur OOP

- Contoh hubungan **project** dengan **class** seperti **Tumbuhan** dengan **taxonomy Tumbuhan**, seperti **Hewan** dengan **taxonomy Hewan**, **seperti Kendaraan dengan jenis-jenis Kendaraan** (motor, mobil, truct, bus).
- Contoh struktur anatomy misal antara **Mobil** dengan **spare-part** nya, contoh lain antara **Manusia** dengan **organ-organ tubuh**-nya.

Class dan Anatomy-nya

1. attribute
2. method()
3. constructor()
4. objek
5. modifier
6. inner-class

Class

- Class adalah **blue print /template benda-benda homogen** atau yang **dibendakan** yang masih abstrak atau umum untuk mencetak /mewujudkan **suatu** objek yang lebih khusus dan nyata, misalnya **class Manusia** dan **objeknya** budi, bambang, joko, dsb.
- ***a car*** itu class, sedangkan ***the car*** itu objek.
- Semua Class **dapat dikompilasi**, sedangkan Class yg memiliki method main() **dapat di run** seperti program tersendiri.

Latihan membuat Class

- Contoh1: class Buku
- attribute nya: ISBN, judul, harga, jumlah halaman, **penulis, penerbit?**
- method() nya: diketik(), diedit(), dicetak(), diterbitkan(), dijual().

- Contoh2: class Mobil
- attribute nya: noSTNK, nopol, merk, warna, tahun, harga.
- method nya: maju(), berhenti(). mundur(), belok().

- Class Mahasiswa
- attribute nya?
- method() nya?

- makan(), minum(), tidur() adalah method() milik class Mahasiswa?

Dua kelompok Class

- Class **standard** yg disediakan vendor atau komunitas yg telah dimasukkan ke dalam *Aplication Programming Interface* (API) dan siap digunakan programmer.

- Class yg dibuat sendiri oleh programmer disebut class **non standard**.

Template struktur Anatomi Class

```
[modifier1] [modifier2] class NamaClass [modifier3] {
   //classbody;
   0 atau 1 atau n attribute;
   1 atau n constructor();
   1 atau n method();
}
```

Modifier pada anatomi Class sifatnya opsional, digunakan berdasarkan kebutuhan.

attribute

- Apa yang dapat diinformasikan mengenai **identitas** atau **ciri khas** atau **kondisi khas** seluruh/sebagian objek dalam sebuah Class. Artinya suatu attribute milik class/objek.
- Attribute dapat diwariskan dan dapat diberi modifier.

attribute

- Deklarasi attribute diletakkan di dalam classbody namun di luar/di atas method().
- Bentuk umum deklarasi attribute ada dua:
- [modifier1] tipedata namaAttribute;
- [public] [static] final tipedata NAMAKONSTANTA = literal;

method()

- Apa saja **yang dapat dilakukan** atau **dialami** sebagian/semua objek di dalam sebuah class, atau **pekerjaan yang khas** yang menjadi perilaku sebagian/semua objek dalam suatu Class.
- Jika suatu method() bersifat ***static*** berarti dapat dilakukan oleh semua objek dalam Class tersebut.
- Method diletakkan dalam classbody, sebaiknya tidak diletakkan dalam method lain.
- Nama method() umumnya berupa pasangan kata **verb-noun**, atau **verb** saja.
- float **getBalance(), doCalculation(), hitungIPK(), setCustomerName(), display().**

method()

- Ada dua kelompok method() yaitu **standard** dan **non standard**.
- Method yang **standard** dibuat bersamaan dengan class standard oleh vendor java. Misal: **setter()** dan **getter()**.

  Method **setter()** untuk memberi literal pada sekumpulan attribute.

  Method **getter()** untuk mengambil informasi pada sekumpulan attribute.
- Method() yang **non standard** dibuat sendiri oleh programmer.

Template/Bentuk Umum method

- [modifier1/2] tipeReturnValue namaMethod([tipe] [parameter]) [throws namaClassEksepsi] {
- methodBody;
- }
- abstract [tipeReturnValue] namaMethod([tipe] [parameter]);
- abstract class MahlukHidup {
- abstract void makan();
  }
- class Manusia extends MahlukHidup {
- makan() {
- System.out.println("cuci tangan dahulu, berdo'a, menggunakan tangan kanan");
- }
- }
- [modifier] [tipeReturnValue] main(String args) {
- methodBody;
- }
- Modifier boleh lebih dari satu (dipisah oleh spasi).
- Pasangan **tipe** dan **parameter** juga boleh lebih dari satu (dipisah oleh koma).

Modifier1 dan Modifier2

Modifier1 dan modifier2mengatur penggunaan Class/komponennya (mengatur Kerjasama antar class dalam sebuah project).

Modifier1:

Yang mengatur **hak akses** **Class pengguna** disebut modifier akses.

Ada dua jenis modifier1, yg pertama adalah kelompok modifier yg **membuka** hak akses class pengguna dan modifier yg **menutup** hak akses class pengguna.

Modifier1 pada anatomi class ditempatkan paling kiri sebelum modifier2.

Modifier2 **memberi sifat** kepada **Class/komponennya** yang disebut modifier sifat/non akses.

Ada 4 keyword yang digunakan sebagai modifier1.

Keyword Modifier1 dan Modifier2

- Modifier1 yg **membuka** **hak akses** class/komponen class pengguna:
  public, protected, default
- Modifier1 yg **menutup** hak akses class/komponen class pengguna:
- **private**
- Modifier2 **memberi sifat** suatu class/komponennya:
  final
  static
  abstract
  super

  synchronized
  native
  storage, transient, volatile

Modifier2 yang memberi **sifat** saling kontradiktif misal **static** dan **abstract** tidak boleh digunakan bersamaan.

Keyword **static** tidak boleh memberi sifat pada interface.
Keyword **super** tdk boleh digunakan pada method() **static**.

Modifier1 (Hak Akses)

- Modifier ini untuk menentukan luas hubungan suatu Class/Interface/komponen Class yang **diakses** oleh class pengguna **yang mengakses**.
- Perhatikan luas wilayah modifier akses berikut ini:

Daftar wilayah Akses

Wilayah Class/Interface/method() yang mengakses	Keyword dalam modifier1 pada Class/Interface/method() yang diakses			
	public	protected	default	Private
di Class yang sama	√	√	√	√
Beda Class, di package yg sama	√	√	√	x
Beda Class, beda package, di Class turunan/implementasi	√	√	x	x
Beda Class, beda package, tidak di Class tururnan /implementasi	√	x	x	x

public

- Menyatakan bahwa class/interface/method() /attribute/inner class yang ditandai dengan keyword ***public*** boleh ***diakses/diimplementasikan*** class pengguna dimana pun.

protected

- Menyatakan bahwa class/interface/method() /attribute/inner class yang ditandai dengan keyword ***protected*** ini boleh ***diakses/diimplementasikan*** oleh **class pengguna** yang berada di dalam ***package*** yang sama atau ***turunannya***.

default

- Menyatakan bahwa class/interface/method() /attribute/inner class yang *tidak* ditandai dengan keyword modifier akses apa pun ini boleh *diakses/diimplementasikan* oleh class pengguna yang berada di dalam *package* yang sama.

private

- Menyatakan bahwa **method()** atau **attribute** yang ditandai dengan keyword *private* ini boleh *diakses/diimplementasikan* **method()** yang berada hanya di dalam *Class* yang sama.

Modifier2 (sifat/adjective)

- Setidaknya ada sembilan keyword penting dalam modifier2 yang bertugas **memberi sifat** pada suatu Class/komponen Class :
- **final, static, abstract, synchronized, native, transient, dan volatile**.
- Secara Bahasa misal:
- **small white home**
- Rumah yang **kecil** dan berwarna **putih** memberi sifat pada rumah.

final (untuk class, method() dan attribute)

- **Class** yang diberi keyword sifat **final**, artinya Class tersebut sudah tidak dapat menurunkan class pengguna. Dengan kata lain sudah tidak dapat di **extends** sub class sebagai pengguna.
- **method()** yang diberi keyword **final** tidak dapat di override class pengguna.
- **attribute** yang diberi keyword **final** maka isinya tidak dapat diubah selama waktu running.

static (sifat pada attribute dan method())

- **Attribute** dan **method()** ada dua jenis, yaitu milik Class atau milik objek.
- **Attribute** atau **method()** yang ***tidak*** diberi keyword static ini adalah non static, artinya milik objek dan diakses ***hanya*** melalui objeknya.
- **Attribute** atau **method()** yang diberi keyword static artinya milik Class, sehingga diakses melalui **Class nya tersebut**, semua objek dalam Class tersebut memiliki sifat yang sama.
- **Method()** yang static tidak boleh di override dan **attribute** yang static boleh tidak diinisialisasi oleh programmer, namun akan diberi nilai default oleh JVM.
- Sehingga **method() yang *non static* tidak dapat memanggil attribute atau method() yang *static*.**
- Contoh kuliah() dan praktikum() adalah method non static berarti milik Sebagian objek Mahasiswa, sedangkan perwalian() adalah method static berarti milik seluruh objek Mahasiswa atau milik class Mahasiswa.
- Contoh lain misal method() standard println(): **System.out.println();**

abstract (untuk class dan method())

- Class yang diberi sifat **abstract** artinya Class tsb murni tanpa objek dan tidak boleh memiiki objek, tidak boleh diinstansiasi.
- Method yang **abstract** di dalam suatu Class harus disempurnakan oleh Class turunannya melalui override.
- Class yang abstract biasanya adalah **root** dari Class-class pengguna dalam struktur Class.
- Konsekuensi penggunaan sifat yg abstract:
- 1. Tidak boleh memiliki constructor yg abstract.
- 2. Tidak boleh memiliki tambahan sifat **static**.
- 3. Tidak boleh memiliki tambahan sifat **private**.

synchronized (khusus modifier method)

- Pada lingkungan multithread, dimungkinkan lebih dari satu jalur eksekusi yang berjalan dalam project yang sama, namun harus diatur bahwa dalam selang waktu tertentu hanya ada satu jalur eksekusi yang diijinkan di **method** yang synchronized (eksekusi dilakukan secara mutual ekslusive).

native (khusus modifier method)

- Keyword modifier2 ini untuk mengakses method (procedure atau pun function) yang ditulis dalam bahasa C++.

- Seperti method() yang abstract, method() yang bersifat ***native*** hanya berupa prototype, implementasi method() yang bersifat ***native*** ini berada di file external namun dalam package/folder yang sama.

transient (khusus modifier attribute)

- Dalam OOP ada konsep serialisasi yaitu kemampuan untuk mentransformasikan objek menjadi suatu stream, sehingga objek dapat ditransfer dari suatu aplikasi ke aplikasi lainnya. Prinsip ini digunakan dalam aplikasi client-server.
- Salah satu ketentuan serialisasi adalah saat objek itu sedang ditransformasikan menjadi stream, tidak boleh ada perubahan nilai attribute suatu objek, namun bila suatu attribute bersifat transient artinya boleh berubah.

volatile (khusus modifier attribute)

- Dalam manajemen thread, nilai suatu attribute yang sering diakses thread, dapat disimpan menjadi cache value, sehingga tidak perlu selalu merujuk ke lokasi memori aslinya.
- Artinya attribute tersebut bersifat *volatile*, karena nilainya rentan berubah bila diakses oleh lebih dari satu thread.